

```
"""
```

```
Cooper River Trading Co. –  
Appraze POS Checkout
```

```
-----  
-----
```

```
Different from billing.py's  
subscriber paywall on purpose:  
that one uses a  
fixed-price Payment Link (same  
price every time, for app  
access). A POS  
sale is a DIFFERENT dollar amount  
every single time, so a single  
static  
Payment Link can't cover it –  
this creates a one-off Stripe  
Checkout  
Session per sale instead, with  
the amount set at creation time.
```

```
Still zero card handling in this  
app: the customer is redirected  
to a
```

Stripe-hosted page and enters their own card there, exactly like a Payment Link. Same security model, just via the API endpoint that supports a dynamic amount rather than one pre-made fixed price.

```
"""
```

```
from dataclasses import dataclass
```

```
import requests
import streamlit as st
```

```
from billing import
verify_checkout_session # reuse
the same read-only status check
```

```
def _secret_key() -> str:
    key =
    st.secrets.get("STRIPE_SECRET_KEY")
    if not key:
        raise
```

```
RuntimeError("STRIPE_SECRET_KEY
is not set in Streamlit
secrets.")
    return key
```

```
@dataclass
class POSCheckoutResult:
    success: bool
    checkout_url: str = ""
    session_id: str = ""
    error: str = ""

def
create_pos_checkout(amount_dollar
s: float, description: str,
customer_email: str = "") ->
POSCheckoutResult:
    """
    Creates a one-off Stripe-
hosted checkout page for a
specific sale amount.
    Returns the URL to show/send
to the customer (or open on a
shared device
```

```
        for a tap-to-pay-style in-
person handoff).
    """
    if amount_dollars <= 0:
        return
    POSCheckoutResult(False,
error="Amount must be greater
than $0.")

    app_url =
st.secrets.get("APP_URL",
"" ).rstrip("/")
    # These redirect URLs are
mostly a nice-to-have: if APP_URL
is set and the
    # SAME device completes
payment (e.g. handed to the
customer and back),
    # the app will land back here
automatically. Either way, the
"Check
    # Payment Status" button in
the POS tab works regardless of
device.
    success_url = f"{app_url}/?
pos_session_id=
```

```
{{CHECKOUT_SESSION_ID}}" if
app_url else
"https://example.com/?pos_paid=1"
    cancel_url = f"{app_url}/" if
app_url else
"https://example.com/?
pos_cancelled=1"

    try:
        amount_cents =
int(round(amount_dollars * 100))
        payload = {
            "mode": "payment",
            "line_items[0]
[quantity]": 1,
            "line_items[0]
[price_data][currency]": "usd",
            "line_items[0]
[price_data][unit_amount]":
amount_cents,
            "line_items[0]
[price_data][product_data]
[name]": description or "Cooper
River Trading Co. item",
            "success_url":
success_url,
```

```

        "cancel_url":
cancel_url,
    }
    if customer_email:

payload["customer_email"] =
customer_email

    resp = requests.post(

"https://api.stripe.com/v1/checko
ut/sessions",
        headers=
{"Authorization": f"Bearer
{_secret_key()}"},
        data=payload,
        timeout=20,
    )
    resp.raise_for_status()
    data = resp.json()
    return
    POSCheckoutResult(True,
checkout_url=data["url"],
session_id=data["id"])
    except
requests.exceptions.HTTPError as

```

```

e:
    detail =
e.response.text[:200] if
e.response is not None else
str(e)
    return
POSCheckoutResult(False,
error=f"Stripe error: {detail}")
    except Exception as e:
        return
POSCheckoutResult(False,
error=f"connection error: {e}")

def
check_payment_status(session_id:
str) -> bool:
    """Read-only poll – call
whenever the admin wants to
confirm a pending sale paid."""
    return
verify_checkout_session(session_i
d).paid

```